

RBE 3001 A24 Team 3 Final Report

David A. Fitzgerald, Evan H. Seki, and Max L. Inman.

Abstract— This project implemented functionality created during the previous four labs, and more, to operate a pick-and-place system to sort objects by color. A system that reliably locates and identifies balls of four known colors and a robot that reliably tracks locations were created for this project. Robotic control is a sum of simple procedures that yields a complex result. This project is the complex result from the simple procedures generated during the labs prior. As future robotic engineers, it is crucial to understand how to break a complex problem down into simple, digestible tasks like done in this project and course.

I. INTRODUCTION

The purpose of this project was to implement computer vision and trajectory generation to command a robotic arm to sort balls by color via a pick-and-place system. These principles were supported by other work done in the previous labs, namely forward kinematic derivation, inverse kinematic derivation, and low-level robotic command. The project implemented computer vision using a fisheye webcam to obtain a snapshot of a checkerboard field where balls of either red, green, orange or yellow would be placed for the robot to pick up and sort. Distortion minimizing techniques, both built-in MATLAB and programmed, were used to obtain the world coordinates of the balls in the frame of the robot. Cubic trajectory planning was used to control the robotic arm along a smooth path as the balls were sorted.

II. METHODOLOGY

The Open Manipulator-X arm used for this project had 4 joints controlling position and 1 joint controlling end-effector status. Therefore, it was crucial to organize the position of the robot using reference frames to make tasks further down the road approachable. Robotic reference frames were assigned in accordance with Denavit-Hartenburg convention. Similar to the scheme given in Robot.m to control the position of the joints, the angle of the joints was chosen to be positive when the arm curled in. This dictated the orientation for the Z axes for joints 2, 3 and 4. A similar choice was made for joint 1 and the Z axis was chosen to point upward as shown in the figure below.

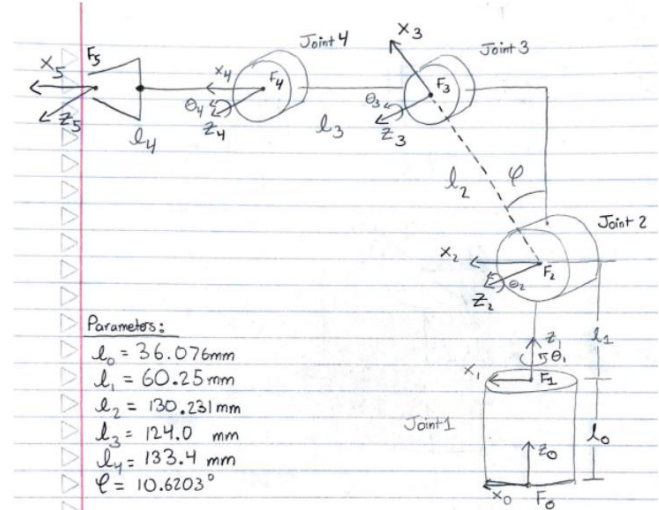


Figure 1. Frames assigned to Open Manipulator X.

The convention established in Figure 1 allowed for the derivation of a DH table to organize the constants and variables found in the positioning of the arm. The parameters in Figure 1 are implemented directly or basic trigonometry was used to derive them from given values in appendix 1.

Transform	DH Parameters			
	Theta	D [mm]	A [mm]	Alpha
1 -> 2	θ_1	60.25	0	-90°
2 -> 3	$\theta_2 + 79.38^\circ$	0	130.231	0
3 -> 4	$\theta_3 - 79.38^\circ$	0	124.0	0
4 -> 5	θ_4	0	133.4	0

Table 2. DH Parameters for Arm.

The parameters in Table 2 allowed for the forward kinematic derivation for the robot arm. These calculations gave the ability to know the X, Y, Z world position of the end effector when given the positions of the joints of the robot. Each row in Table 2 was used to calculate a homogeneous transform matrix from the frames listed in the "Transform" Column, using the matrix in appendix 2. The transform matrix from the ground frame $\{F_0\}$ to frame 1 was derived by inspection and found to be:

Transform Matrix			
1	0	0	0
0	1	0	0
0	0	1	36.076 mm
0	0	0	1

Table 3. Homogeneous Transform Matrix from F_0 to F_1

These matrices were post multiplied to get the pose of the end effector in F_0 of the robot in terms of the joint angles θ_1 through θ_4 . Implemented in MATLAB, this was done symbolically when a new robot object was created. The MATLAB function *matlabFunction* was used to create an anonymous function that allowed for very fast substitution of joint angles into this symbolic matrix. This was wrapped in the Robot method *fk3001*, which took a vector of joint angles as an argument and ran the *matlabFunction* generated in the constructor.

Forward velocity kinematics refers to the process of calculating the angular and linear velocities of a robot end effector. This occurs through use of a Jacobian; a $6 \times N$ matrix relating joint velocities to an end effectors angular and Linear Velocity. The Jacobian is composed of two parts the position (J_p) and orientation (J_o).

$$J_p = \begin{bmatrix} \frac{\partial x}{\partial \theta_1} & \frac{\partial x}{\partial \theta_2} & \frac{\partial x}{\partial \theta_3} & \frac{\partial x}{\partial \theta_4} \\ \frac{\partial y}{\partial \theta_1} & \frac{\partial y}{\partial \theta_2} & \frac{\partial y}{\partial \theta_3} & \frac{\partial y}{\partial \theta_4} \\ \frac{\partial z}{\partial \theta_1} & \frac{\partial z}{\partial \theta_2} & \frac{\partial z}{\partial \theta_3} & \frac{\partial z}{\partial \theta_4} \end{bmatrix}$$

Figure 2. Calculation of J_p .

Demonstrated in Figure 2 above is the time derivative approach to calculate the upper half of the Jacobian, J_p a 3×4 matrix. The number of columns for J_p represented the number of joints for the robot. Each row is defined by a different position of the end effector in task space; row one follows the X position, row two the Y position, and row three the Z position. The partial derivative of each was then taken with respect to each of the joints. In column one the partial derivative is taken with respect to joint 1, column two with respect to joint 2, column three with respect to joint 3, and column four with respect to joint 4.

Similar to J_o , J_p is also a 3×4 matrix with each column representing a joint of the robot. To determine the lower half of the Jacobian, J_o , the intermediate transformation matrices were calculated. Each column of the J_p is the last column of the intermediate rotation matrices Z_i .

Forward velocity kinematics was implemented once the final Jacobian was calculated by creating a 6×4 matrix with J_p stacked on top of J_o . Multiplying the Jacobian with a $N \times 1$ vector of joint velocities produced a 6×1 vector of the end effector's linear and rotational velocities.

The implementation of forward kinematics in MATLAB allowed for the implementation of inverse kinematics for the robot. This was done in the Robot.m method *ik3001*. To simplify the implementation, the position and orientation of the end effector were decoupled and only the position of the end effector was considered. This allowed the solution to be split into two parts: the derivation of the position of joint 4, and the derivation of the joint angles θ_1 through θ_3 . The first calculation represents the kinematic decoupling.

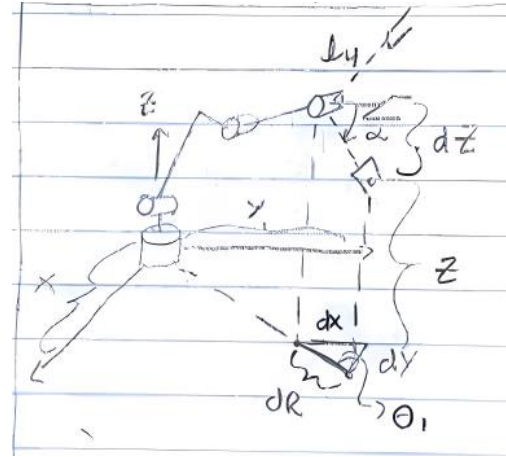


Figure 3. Decoupling of End Effector.

The first step in the decoupling was to calculate the distance along the axis of the robot (r , Figure 4), and the Z axis to subtract the length of the end effector by. This was done by projecting the length of the end effector onto those two axes. As α was defined as the angle of depression from the horizontal, this meant that:

$$dZ = 133.4 * \sin(\alpha)$$

$$dR = 133.4 * \cos(\alpha)$$

The distance dR was projected onto the X and Y axes to calculate the distance along X and Y to subtract the end effector position by to get the position of joint 4. To do this, however, θ_1 first had to be calculated. θ_1 through θ_3 are defined in the below figure.

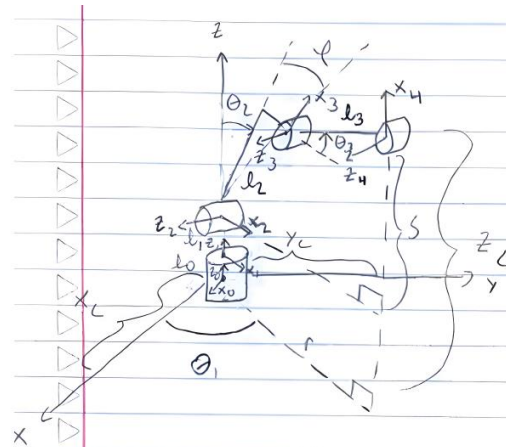


Figure 4. Definition of Joint Angles of Decoupled Robot

The value of θ_1 was calculated by the equation:

$$\theta_1 = \text{atan2}(X, Y)$$

Where atan2 is built into MATLAB and X and Y are given to the method *ik3001* as a part of the argument. With this implementation, however, one of the technically possible solutions for θ_1 was discarded. Therefore, the inverse kinematics solution was technically limited. However, the limited solution for θ_1 ensured that the robot always faced towards the position in worldspace, which was desired for the application. With θ_1 calculated, dR was then decomposed into X and Y components by the following:

$$dX = dR * \cos(\theta_1)$$

$$dY = dR * \sin(\theta_1)$$

Then, the end effector was decoupled and the position of joint 4 was calculated as:

$$X_c = X - dX$$

$$Y_c = Y - dY$$

$$Z_c = Z - dZ$$

As shown in Figure 4, X_c , Y_c , and Z_c define the position of the fourth joint in task space.

Next, angles θ_2 and θ_3 were solved for. Another view of the robot with just θ_2 and θ_3 was generated and is shown below.

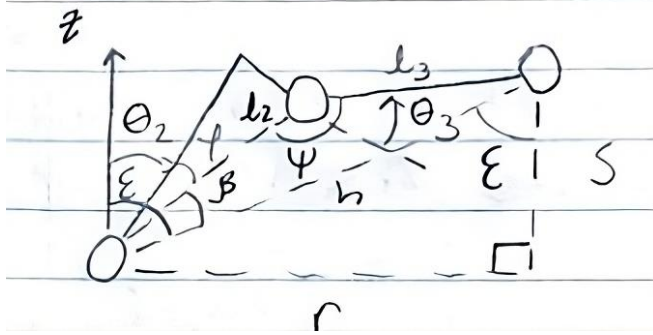


Figure 5. Close-up of θ_2 and θ_3 .

First, intermediate values were defined to aid in kinematic derivation. These values were defined as

$$r = \sqrt{X_c^2 + Y_c^2}$$

$$s = Z_c - (l_0 + l_1)$$

$$h = \sqrt{r^2 + s^2}$$

The length r was calculated as the hypotenuse of the triangle created by X_c and Y_c . The length s was calculated as the Z position of joint 4 subtracted by the distance between the F_0 and joint 2. The length h was calculated as the hypotenuse of the triangle generated by r and s . From this point, intermediate angles were used to calculate the values for θ_2 and θ_3 . As shown in Figure 5, these were defined as:

$$\theta_2 = \epsilon - \beta - \varphi$$

$$\theta_3 = 90 + \varphi - \psi$$

These intermediate angles were calculated in a specific manner to preserve angle ambiguity. Unlike for the calculation of θ_1 , two angle solutions were desired. This preserved multiple solutions for the joint positions of the robot, as it was unknown at this point in time which combinations of which intermediate angles produced the correct θ values. First, the cosine of ϵ was calculated as:

$$\cos(\epsilon) = \frac{s}{h}$$

Then, the sine of ϵ was derived from its cosine via the Pythagorean formula for sines and cosines (Appendix 3). Therefore, the sine of ϵ was:

$$\sin(\epsilon) = \pm \sqrt{1 - (\cos(\epsilon))^2}$$

In MATLAB, both solutions for the sine of ϵ were preserved by multiplying the output of the function *sqrt* by a 2 x 1 vector of 1 and -1.

Next, the MATLAB method *atan2* was used to calculate the two possible angles for ϵ . This was implemented as:

$$\epsilon = \text{atan2}(\sin(\epsilon), \cos(\epsilon))$$

The cosine of β was calculated using the law of cosines and was found to be:

$$\cos(\beta) = \frac{h^2 + l_2^2 - l_3^2}{2 * h * l_2}$$

Where l_2 and l_3 are defined in Figure 1. The same strategy as shown for ϵ was followed for β to calculate the two possibilities for it. The angle φ was constant and is shown in Figure 1.

To calculate θ_2 , the combinations of ϵ , β and φ were considered. As there were 2 intermediate angles with two possibilities each, there were four possible combinations for θ_2 .

To calculate θ_3 , first the intermediate angle ψ was calculated. The cosine of ψ was calculated using the law of cosines to be:

$$\cos(\psi) = \frac{l_2^2 + l_3^2 - h^2}{2 * l_2 * l_3}$$

The two possibilities of ψ were calculated with the same strategy as for ϵ and β . As there was only one intermediate angle calculated with two possibilities, there were two possible solutions for θ_3 .

The derivation of θ_4 was done for each set of possible θ_2 and θ_3 combinations. The derivation of θ_4 was found via observation to be:

$$\theta_4 = \alpha - \theta_2 - \theta_3$$

With one possibility for θ_1 and θ_4 , two possibilities for θ_3 , and four possibilities for θ_2 , there were eight different possibilities for the solution to the inverse kinematics of the robot. These solutions were then filtered by two different criteria.

The first filter removed solutions from the eight possible solutions if they were physically impossible to reach due to joint space constraints. This protected the robot from attempting to reach positions that would destroy the links connecting the joints. The next filter used the Robot method *fk3001* to check the error in task space position between the possible solution and the desired solution. The first solution that was within 1mm of the task space solution was then returned as the decoupled inverse kinematics solution for the robot.

The implementation of inverse kinematics for the robot arm allowed for control of the robot's position in task space. Using a camera, ball locations in task space were obtained using image processing.

The first step in the image processing pipeline was intrinsic camera calibration. This was necessary for the camera to be able to convert the pixel coordinates to positions in task space. MATLAB built-in calibration software took in images of a

known pattern with a known size at different angles to find the focal length, principal point, and geometric of the camera used. These parameters represented the intrinsic calibration parameters.

The calibration in the lab for the camera was done with a 5x11 black and white checkerboard pattern with each square being 25mm x 25mm. Using the Camera Calibration app in MATLAB, 40 images of this pattern were taken from different angles with the camera to determine the intrinsic camera calibration. The app took these images and assigned a point to each intersection of squares in the checkerboard. From this, an XY coordinate system was also determined and placed. Out of these 40 images only the best 20 images were chosen. The criteria for the best images included the checkerboard being entirely in frame, consistency among the XY coordinate system, and a mean error in pixels of less than one.

After the completion of intrinsic camera calibration, extrinsic calibration was completed to translate frames from the camera F_{Cam} to the checkerboard $F_{Checker}$ to the base frame of the robot F_0 , as shown in Figure 5 below. These rotation matrices were multiplied together to get T_{Cam}^{Robot} which converted coordinates in the image frame to coordinates in base frame.

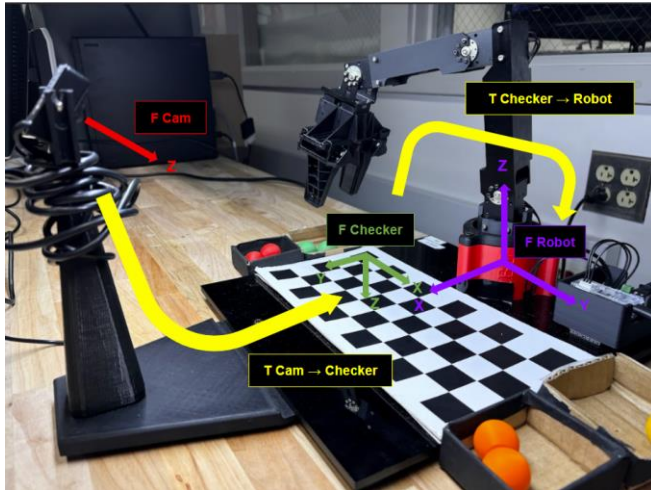


Figure 6. Image of robot with F_{Cam} , $F_{Checker}$, and F_{Robot} shown.

To find the location of colored balls on the checkerboard a mask of the checkerboard was applied to the image to remove everything but the workspace that balls were placed.

The four colors of balls the camera detected were red, orange, yellow, and green. To distinguish these colors in the image the Color Threshold app in MATLAB was used to create a mask for each ball. To do this an image of the masked checkerboard with all four colored balls on it was taken and converted from RGB to HSV color space. This was because it was determined that the hue of the four colors was easier to separate in HSV color space compared to RGB color space. It was also determined that strong, consistent lighting aided in creating masks that separated the balls. To ensure consistent lighting, a bar light was aimed at the checkerboard from top down near the camera. A color mask was then created for each color and saved to a function, which turned an image into a

binary image by replacing all pixels in the input image in the color mask with value 1 (white) and replacing all other pixels with the value 0 (black).

To find the center of each masked ball in each masked image, the function *ballsCent* was created. The function took in a binary image and returned the centroids of all the objects in the image. It did this by first removing any stray pixels that were accidentally collected by using the MATLAB function *imerode*, which removed the outside layer of objects in a see figure 13 in results section for example. However, before this is done *imfill* is used to make sure that any pixels that are missing inside an object do not erode the object from the inside out. The objects remaining after the erode are then put back to their original size by dilating the image with *imdilate*. The objects in the binary image are then edged out and filled in one more time to ensure consistency in centroid calculations. The last step before calculating the centroid is converting the binary image to a label image which assigns pixel values 1:n (where n is the number of objects in the image) to each separate object in an image with the background pixels remaining at 0. Finally, the centroid of the object is calculated with the function *regionprops*, which finds the centroid of each object in a label image.

The pixel coordinates of the centroid are then converted to their true position on the XY plane of the checkerboard.

Due to inherit limitations of the vision system, the value returned from the Camera method *ballsCent* returns the centroid of the projection of the ball of the checkerboard. However, this information was not yet useful for the project as it did not describe the position of the ball in 3D space. Therefore, a Camera method *pixelToBallCoords* was created to organize the deprojection of the ball into the task space.

First, the MATLAB function *pointsToWorld* was used to convert the pixel coordinates from *ballsCent* into world coordinates in the checkerboard frame using the intrinsic camera parameters from the camera calibration. These coordinates represented, essentially, the centroid of the “shadow” of the ball on the board. Therefore, trigonometry was used to interpolate the actual position of the center of the ball, knowing the size of it. First, the angle θ was determined.

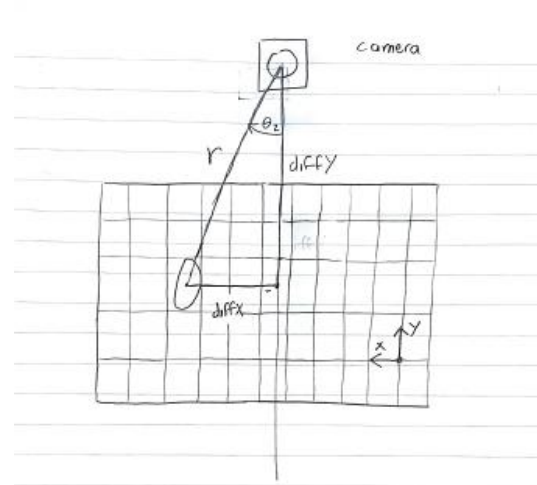


Figure 7. Derivation of θ .

The values diffX and diffY were calculated by converting the planar position of the centroid into the camera frame using extrinsic camera calibration parameters, and were used to calculate θ .

$$\theta = \text{atan2}(\text{diffX}, \text{diffY})$$

Next, a triangle formed between the ball and the camera.

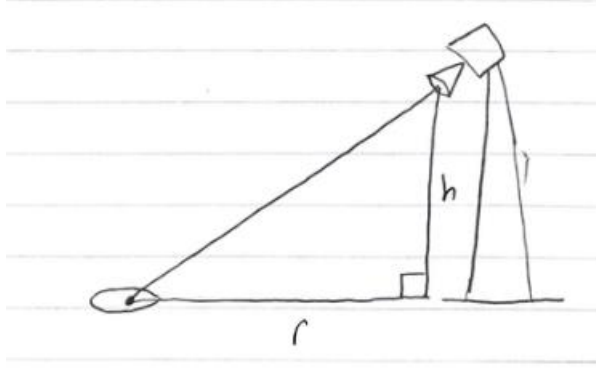


Figure 8. Triangle of Ball Shadow Centroid and Camera

H was measured to be 175mm. The length r was calculated as the hypotenuse of the triangle created by diffX and diffY . The triangle built by r and h in Figure 9 is zoomed in below, with the spherical ball shown.

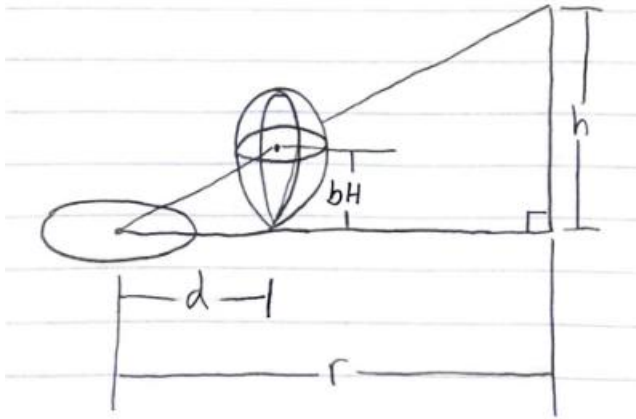


Figure 9. Zoomed in Projection-Ball Camera Triangle

Figure 9 shows the similar triangles created between r and d - bH . The length bH is half of the ball height and was measured to be 11mm. As three out of the four lengths were defined, the fourth length d could be calculated.

$$d = \frac{r * bH}{h}$$

The length d was found to be the distance along r to subtract to get the planar positions of the centroid of the ball. The height of the ball was constant, at bH .

$$\text{ball } X = \text{shadow } X - d * \sin(\theta)$$

$$\text{ball } Y = \text{shadow } Y + d * \cos(\theta)$$

$$\text{ball } Z = 11$$

These coordinates for the ball were in the frame of the checkerboard. For additional functionality they were converted to the base frame of the robot. See Figure 8 below for a diagram of Image processing.

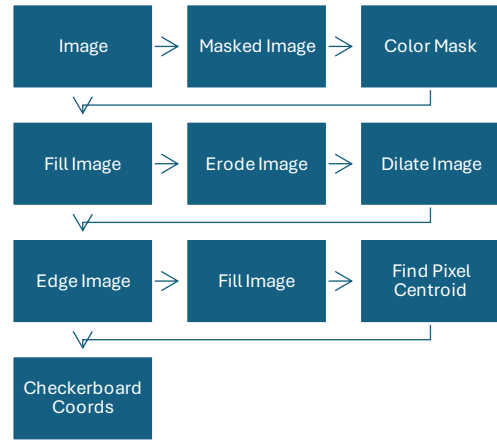


Figure 10. Image processing Pipeline.

III. SYSTEM ARCHITECTURE

fk3001 takes four joint positions in degrees and calculates forward kinematics given an end effectors XYZ position in task space. This function was utilized when the robot current position was required.

ik3001 takes the X, Y, and Z position of the end effector in addition to an alpha which defined the angle of the end effector with respect to the ground. This method outputs the joint angles of the arm. This function was utilized within the *run_Tajjectory* method to calculate desired joint angles on a trajectory path within joint space.

cubic_Traj takes 6 inputs; a starting and ending position, a starting and ending time, and a starting and ending velocity. The method then outputs a 4 x 1 array containing coefficients a_0 through a_3 of the cubic polynomial. These coefficients were used as inputs to the *run_trajectory* method.

run_Trajjectory takes in either a 4 x N of calculated joint trajectory coefficients or a 4 x N of calculated X, Y, Z and alpha trajectory coefficients. This method then uses the input coefficients to move the arm to the new desired location. After the balls were located, *run_Trajjectory* was utilized to navigate the arm into the proper position collect and deposit balls

ballsCent takes in a black and white Binary Image and removes noise so that the remaining image is a clean mask of the desired colored balls. The method then returns an N x 1 struct of 2 x 1 pixel coordinates. These pixel coordinates were used to determine XYZ coordinates with respect to the Robot Frame.

isColorOnBoard takes a string and a masked image. The method then checks if more than 150 pixels of the desired color are present in the image. If so, the method returns true, signifying that the color is present. This method was utilized in our final project runner in a for loop to determine which colored balls were present on the checkerboard.

getImage takes no inputs. However, once called the method takes a snapshot of the board. The camera has a fisheye lens. Before this method outputs the image the distortion from the

lens was removed. This method functions as the camera's 'eyes' allowing the board to be observed.

maskBoard takes in an undistorted image. The method crops out everything around the checkerboard leaving only the board and its contents visible. This method was used before any analysis was performed on the picture.

pixelToBallCoords took a 2x1 vector of pixel coordinates and converted them to XYZ coordinates in the robot frame. This was used to determine the robot's trajectory.

Classes were created to group these functions into different categories. The Robot.M class was used for calculating robot kinematics as well as reading and writing data to the motors. Method such as *fk3001* and *ik3001* were stored here. The Traj_Planner class takes in a robot object and includes the functions for trajectories such as *cubic_Traj* and *run_Trajectory*.

The Camera.m contains the code for image processing and calculating ball coordinates. Methods include: *ballsCent*, *isColorOnBoard*, *getImage*, *maskBoard*, and *pixelToBallCoords*.

IV. PROJECT SCHEME

To implement a pick and place system for the robot a state machine was created to iterate through the different tasks required. See figure 11 below for diagram of state machine.

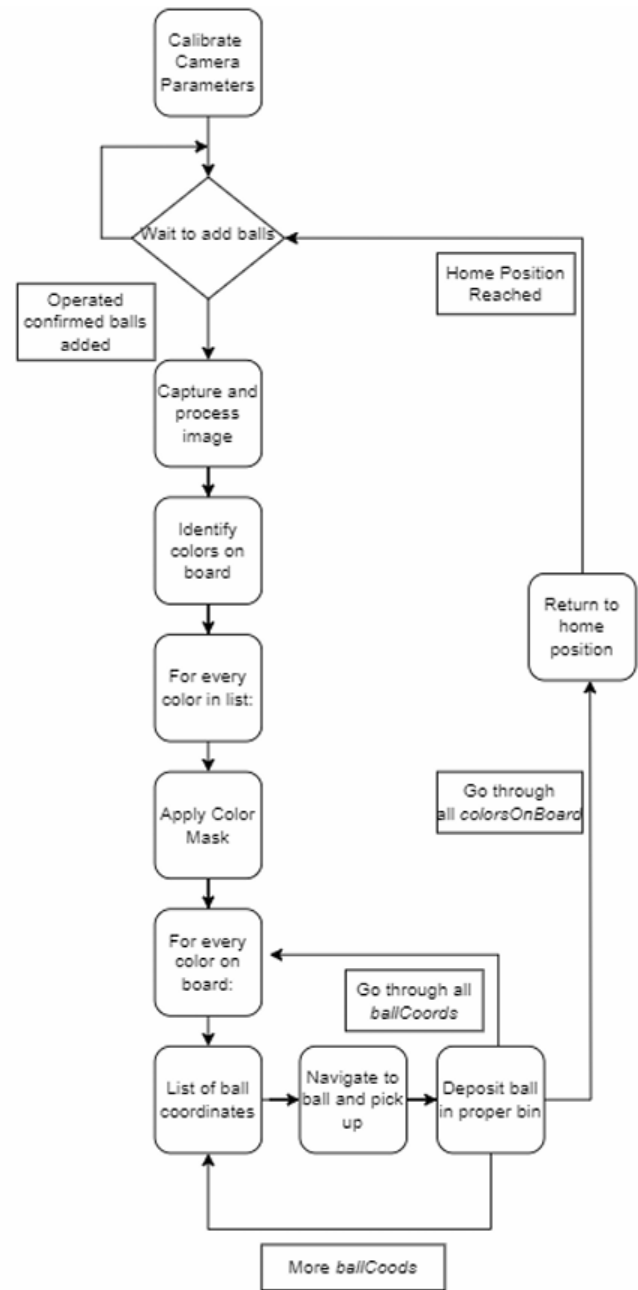


Figure 11. Diagram of State Machine.

As seen in Figure 11. The robot starts by calibrating the camera. This only happens once at the beginning because the pose of the camera does not change. The robot then waits for human input to tell the robot that balls have been added to the checkerboard. Next, the robot captures an image, identifies the colors present on the checkerboard, and applies each color mask to the image to separate the different colored balls. For each color present, the robot finds the centroid coordinates of each object in the robot's frame and then uses those coordinates to create and execute a trajectory to that ball from the robot's current position. The robot then picks up the ball and deposits it in the proper bin given the color. After the ball is deposited it will either go to the next object of the same color if present, go to the next object of a different color if present, or return to the home position if there are no balls remaining. From the home position it will then wait for human input to tell the

robot if more balls have been added and repeat the state machine again.

V. RESULTS

See figures 12-18 below for image processing pipeline.

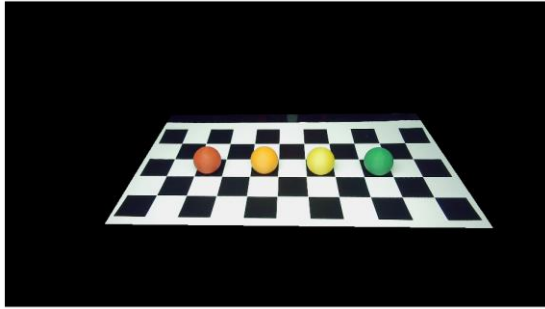


Figure 12. Masked image with balls.

As seen in Figure 12, the image is masked so that only the checkerboard is in frame.

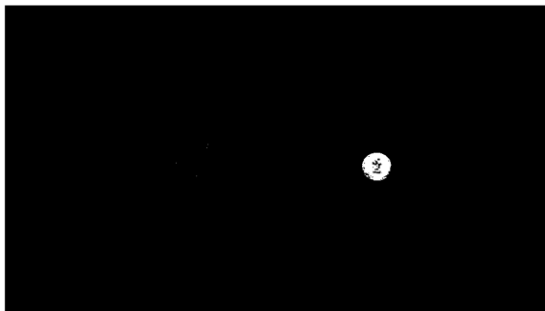


Figure 13. Image with green mask filter.

As seen in Figure 13, the green color mask was applied to the image in figure 12 which returns an image with all pixels that fit into the green color mask to 1 (white) and all other pixels to 0 (black).



Figure 14. Filled Image 1.

As seen in Figure 14, the matlab function *imfill* was applied to remove any holes in the ball.

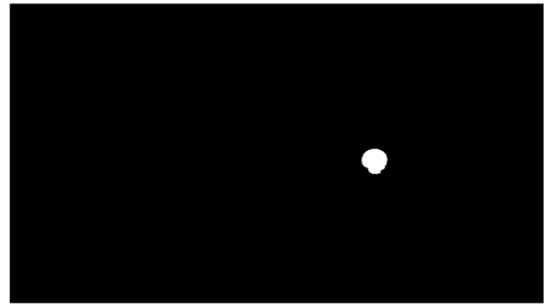


Figure 15. Eroded Image.

As seen in Figure 15, the MATLAB function *imerode* was applied to remove any stray pixels.

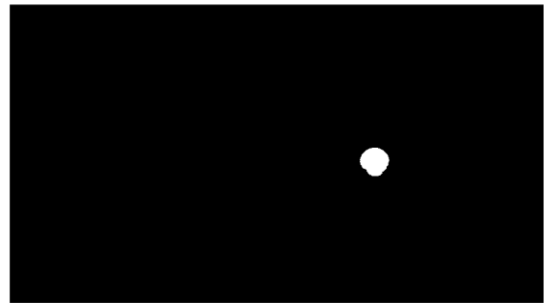


Figure 16. Dilated Image.

As seen in Figure 16, the MATLAB function *imdilate* was applied to revert the objects size back to what it was before erosion.

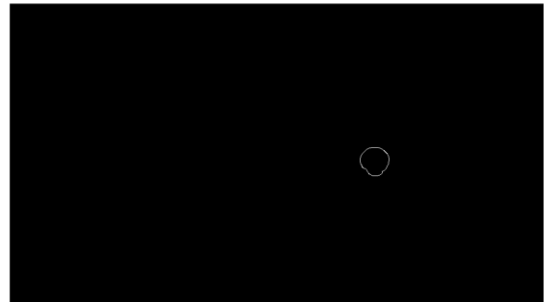


Figure 17. Edged Image.

As seen in Figure 17, the MATLAB function *edge* was applied to create an outline of the ball.

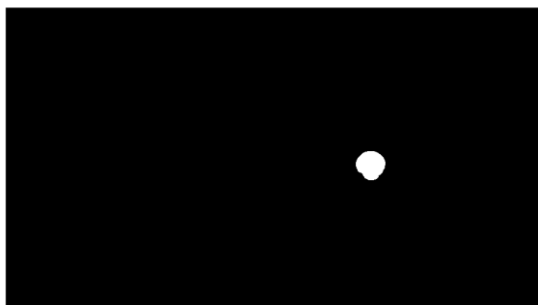


Figure 18. Filled Image 2.

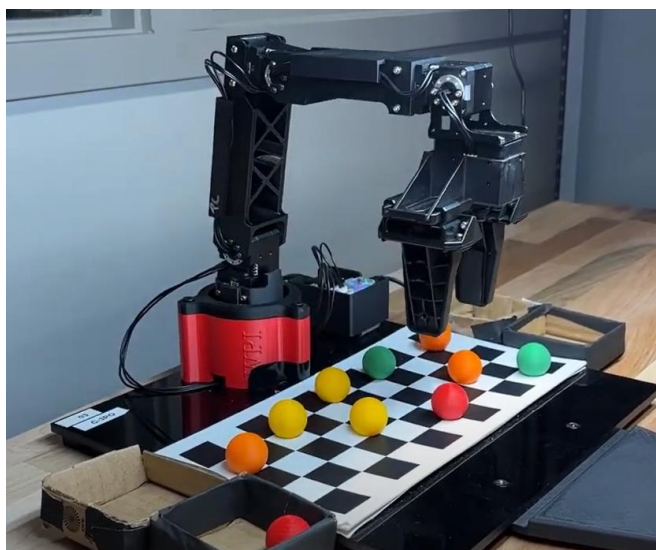


Figure 19. Robot Located Above Ball

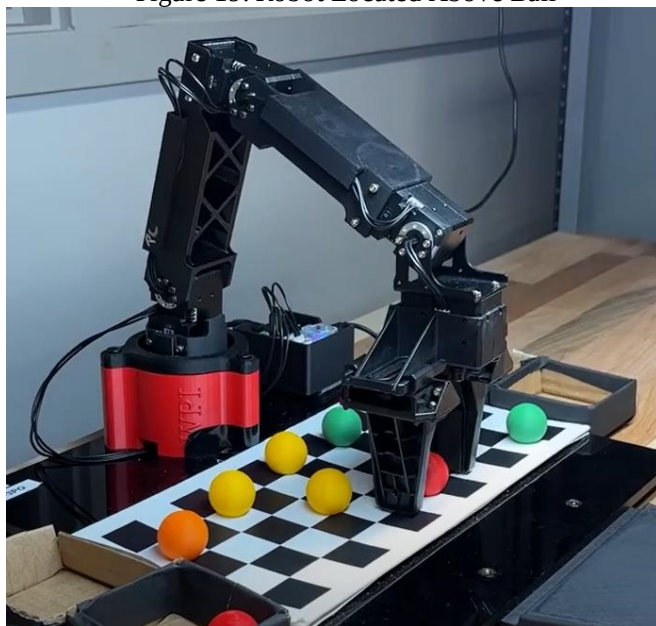


Figure 20. Robot Grabbing Ball



Figure 21. Robot Depositing Ball

As seen in Figure 18 the MATLAB function *imfill* was applied once again to fill in the outline of the ball from figure 17.

The robot can successfully identify each color ball and determine the coordinates of the ball in the frame of the robot within $\pm 0.5\text{cm}$ in the X and Y direction 10/10 times. The robot places the ball in the correct container 10/10 times. The robot has additional functionality of being able to sort multiple balls of the same color. The robot is also able to sort balls if they are added to the board while the robot is running. A video of the robot executing the sorting program can be seen in appendix 4.

VI. DISCUSSION

The image processing pipeline went under many iterations. Eroding too much had a risk of removing objects entirely from the image, while not eroding enough caused unwanted stray pixels to remain in the image. The most difficult task was ensuring consistent lighting when creating the color masks. This is why a bar light was added above the checkerboard to ensure consistent lighting.

The final sorting algorithm is efficient and consistent. The error of $\pm 0.5\text{cm}$ in the X and Y direction is acceptable considering the ball is 2.5 cm in diameter and the gripper has a pickup range of 3.0cm x 5.0 cm.

Some limitations of the robot are if the balls are too close to each other, the gripper will collide or miss balls as it attempts to collect them. If balls of the same color are too close to each other, the centroid calculation will be at the center of the two balls which results in a miss. The balls must be entirely within the checkerboard, otherwise they will be out of reach of the arm in the far corners because the program keeps the arm of the manipulator aimed straight down during ball pickup.

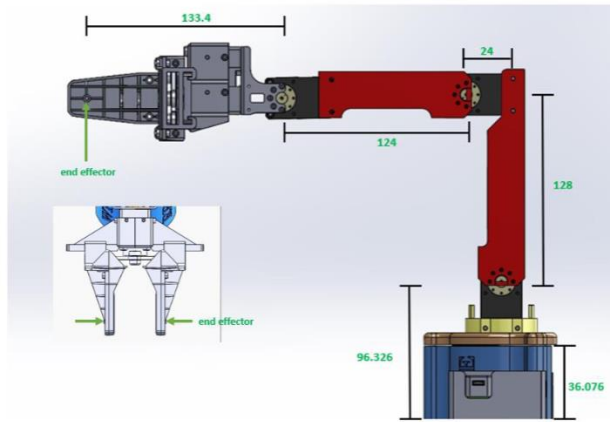
VII. CONCLUSION

By employing computer vision for object detection and trajectory planning for robotic arm control, the system demonstrated the ability to reliably identify, locate, and manipulate balls of four distinct colors: red, green, orange, and yellow. The project demonstrates the role of calibration, both intrinsic and extrinsic, in computer vision, ensuring accurate mapping of pixel coordinates to real-world locations. Using homogeneous transformation matrices and kinematic equations to apply forward and inverse kinematics was crucial for precise robot movement, while image processing allowed for accurate identification of object positions in the task space. This results in the successful implementation of a pick-and-place system to sort objects by color.

The success of the project emphasizes the importance of iterative development in robotics, where complex systems can be constructed through a series of simpler, modular components. The integration of computer vision, kinematic control, and trajectory planning illustrates the interdisciplinary nature of modern robotic systems. These methods can be applied to accommodate more complex tasks in industry. This project serves as a strong example of how foundational principles in robotics can be applied to solve real-world challenges in automation and control.

APPENDIX

1.) Given Dimensions of Robot



2.) Calculation of Homogeneous Transform Matrix for Row of DH Table:

$$T_i^{i+1} = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i\cos(\alpha_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i\sin(\alpha_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.) Pythagorean Identity of Cosines and Sines

$$\sin^2(\theta) + \cos^2(\theta) = 1$$

4.) Link to video:

<https://www.youtube.com/watch?v=9P7hzNfme7k>

5.) Link to code:

https://github.com/RBE3001-A24/RBE3001_A24_Team_3/releases/tag/Lab5Final

ACKNOWLEDGMENTS

Thank you to the SAs for endless support in the lab. Thank you to the TAs and Grader for support in learning course material. Thank you to Professor Agheli for inspiring curiosity in the classroom and applying knowledge in the lab.

CONTRIBUTIONS

Contribution	Team Member(s)
CODING	DF, ES, AND MI
EXPERIMENTING	DF, ES, AND MI
ANALYZING RESULTS	DF, ES, AND MI
PERFORMING WRITE-UPS	DF, ES, AND MI
CREATING VIDEO	ES